

## 13 — Stati, rotte e il tasto Indietro

**La domanda a cui risponde:** quando una schermata può mostrare tre cose diverse, dove si tiene l'informazione su *quale* delle tre sta mostrando? E perché la risposta sbagliata rompe un pulsante che non hai scritto tu — l'Indietro del browser?

### Il fatto

I mazzi salvati non avevano una casa. Erano un `<ul>` appeso in fondo a due schermate diverse: sotto il wizard in “Crea mazzi”, e sotto il costruttore manuale in “Il mio mazzo”. Per arrivarci si scorreva oltre tutto ciò che serve a *crearne* di nuovi.

Il codice lo ammetteva da solo. Dopo ogni salvataggio c'era questo:

```
// vista-mazzi.js, prima
salvati.scrollToView({ block: 'nearest', behavior: 'smooth' });
```

Uno `scrollIntoView` di consolazione: l'elenco era così in basso che senza portarcisi il salvataggio sembrava non riuscito. **Quando l'interfaccia ha bisogno di uno scroll automatico per farsi capire, il problema non è lo scroll.**

E aprire un mazzo salvato non portava da nessuna parte: si ridisegnava tutto nella stessa pagina, `location.hash` non cambiava, e il tasto Indietro — non avendo niente da annullare — usciva dalla schermata invece di tornare all'elenco.

### Lo stato invisibile

Il difetto ha un nome preciso: **stato applicativo tenuto fuori dall'URL**.

```
// Prima: la visibilità era una conseguenza di chi aveva chiamato cosa.
function disegnaPiano(piano, opzioni) {
  zonaWizard.hidden = true;
  risultato.hidden = false;
  // ...
}
```

Chi guarda quel codice non sa dire, leggendolo, quali stati esistono: lo scopre inseguendo chi chiama `disegnaPiano`, chi chiama `ricomincia`, chi tocca `.hidden` altrove. Lo stato c'è — sono tre configurazioni di `hidden` — ma non ha un nome, non ha un posto, e nessuno lo può interrogare.

L'URL invece è **un posto**: pubblico, condivisibile, con una pila di cronologia che il browser gestisce gratis. Spostarci lo stato non è “usare il routing”: è dare un nome a qualcosa che ce l'aveva già in forma implicita.

```
#mazzi                → la libreria
#mazzi/nuovo          → il wizard
#mazzi/risultato      → i mazzi appena generati (non ancora salvati)
#mazzi/<id>            → un salvataggio riaperto
#personalizzato/<id> → quel salvataggio ricaricato nel costruttore
```

Con questo, tutte le funzioni che prima toccavano `hidden` smettono di farlo, e resta **un solo punto** che decide cosa si vede:

```
// src/app/vista-mazzi.js
async function mostraStato(parametro) {
  const nelWizard = parametro === 'nuovo';
  const nelDettaglio = parametro !== '' && !nelWizard;

  libreria.hidden = parametro !== '';
  zonaWizard.hidden = !nelWizard;
  risultato.hidden = !nelDettaglio;
```

```
// ...  
}
```

disegnaPiano() adesso *scrive dentro* #risultato-mazzi e basta. Se decidesse anche la visibilità, un piano ridisegnato dopo una sostituzione di carta riaprirebbe il dettaglio anche stando in un'altra schermata.

## Differenza con Angular

In Angular il router è un modulo con le sue Route[], i suoi RouterOutlet, le sue guardie. Qui il router intero sono 60 righe (src/app/viste.js) e la parte nuova è **una riga di parsing**:

```
export function spezzaFrammento(frammento) {  
  const pulito = String(frammento ?? '').replace(/~#/ , '');  
  const barra = pulito.indexOf('/');  
  if (barra === -1) return { nome: pulito, parametro: '' };  
  return { nome: pulito.slice(0, barra), parametro: pulito.slice(barra + 1) };  
}
```

indexOf e non split('/'): l'id di un mazzo salvato è una data ISO, e split su un parametro che contenesse una barra lo taglierebbe a metà. Solo la **prima** barra separa.

Perché il frammento (#mazzi) e non la History API (/mazzi)? Perché su GitHub Pages non c'è un server da configurare: aprire /pokedeck-famiglia/mazzi darebbe 404, mentre il frammento non lascia mai la pagina. È lo stesso motivo per cui i vecchi single-page Angular usavano HashLocationStrategy.

## La delega degli eventi non è un'ottimizzazione

Prima, i pulsanti di navigazione si collegavano uno per uno all'avvio:

```
for (const collegamento of document.querySelectorAll('[data-vai]')) {  
  collegamento.addEventListener('click', () => vaiA(collegamento.dataset.vai));  
}
```

Funzionava finché tutti i data-vai erano nell'HTML statico. Il pulsante “< I miei mazzi” del dettaglio invece **nasce a ogni ridisegno**: querySelectorAll all'avvio non lo vede, e restava muto. Un ascoltatore sul document che risale con closest copre anche ciò che non esiste ancora:

```
document.addEventListener('click', (evento) => {  
  const collegamento = evento.target.closest('[data-vai]');  
  if (collegamento) vaiA(collegamento.dataset.vai);  
});
```

Non è “meno listener quindi più veloce” — è che **la delega è l'unico modo di ascoltare elementi futuri**. In Angular non ci si pensa perché il template si riassocia da solo; senza framework, il DOM che riscrivi è DOM nuovo, e i vecchi ascoltatori se ne vanno con quello vecchio.

## <details>: la fisarmonica che non devi scrivere

Il dettaglio di un piano faceva attraversare **sette** blocchi di prosa prima della prima carta del primo mazzo. Nessuno di quei blocchi è inutile — spiegano perché ci sono evoluzioni giocate come Base, quanto sono forti i mazzi, se sono pari fra loro — ma nessuno è ciò che si cerca **riaprendo** un mazzo salvato. Lì si cercano le carte da pescare.

La soluzione è un elemento HTML che esiste dal 2011:

```
<details class="sezione-piano">  
  <summary>Regole della casa (2)</summary>  
  <div class="sezione-piano-corpo">...</div>  
</details>
```

<details> porta con sé apertura, chiusura, tastiera, ruolo ARIA e stato open interrogabile dal CSS — **senza una riga di JavaScript**. Il progetto ha già una fisarmonica scritta a mano in `vista-regole.js`, ma lì serviva tenerne aperta *una sola per volta*, che è un comportamento che l'elemento nativo non offre. Qui non serve, quindi non si scrive.

Il titolo conta le cose che contiene: Regole della casa (2), non Regole della casa. **Un titolo che non dice quanto c'è dentro costringe ad aprire per sapere se valeva la pena.**

E quella con le regole si apre da sola:

```
sezioneRichiudibile(  
  `Regole della casa${piano.regole.length ? ` (${piano.regole.length})` : ''}`,  
  fogliaRegole(piano.regole),  
  { aperta: piano.regole.length > 0 },  
)
```

Sono istruzioni da leggere prima di giocare, non un approfondimento. Mentre l'avviso dei mazzi incompleti resta **fuori** da qualunque sezione: *un errore che bisogna aprire per scoprire non è un errore*.

## Perché la stampa richiede JavaScript oltre al CSS

Qui c'è la trappola vera. Il foglio stampato deve contenere **tutto**: liste delle carte, regole della casa, proxy. Ma metà del contenuto ora sta dentro <details> chiusi.

L'istinto è risolverlo in CSS:

```
@media print {  
  .sezione-piano > summary { display: none; }  
  /* ...e il corpo? */  
}
```

Non basta. Il contenuto di un <details> chiuso non è “nascosto con `display: none`” da una regola che puoi sovrascrivere: il comportamento è definito dallo stato `open` dell'elemento, e i motori lo implementano internamente (`content-visibility`, uno slot che non viene reso). Forzarlo dal foglio di stile funziona su alcuni browser e non su altri — cioè, in pratica, non funziona.

L'unico modo affidabile è cambiare **lo stato**, non l'aspetto:

```
intestazione.querySelector('#bottone-stampa').addEventListener('click', () => {  
  for (const d of risultato.querySelectorAll('details')) d.open = true;  
  window.print();  
});
```

Sincrono e prima di `window.print()`, che blocca finché il dialogo non si chiude: quando la finestra di stampa apparecchia la pagina, le sezioni sono già aperte.

Il CSS di stampa fa il resto, cioè trasformarle in capitoli:

```
/* src/ui/stile/stampa.css */  
.sezione-piano { border: none; margin: 0 0 6mm; }  
.sezione-piano > summary { font-size: 14pt; font-weight: 700; list-style: none; }  
.sezione-piano:has(.foglio-regole) { break-before: page; }
```

Stessa lezione, un livello più giù, con le schede dei mazzi:

```
mazzo-generato[hidden] { display: block !important; }
```

A schermo si vede un mazzo per volta; sul foglio ci vanno tutti. Qui il CSS *basta*, perché `hidden` è un attributo il cui effetto è una normale regola `display: none` dell'user agent — sovrascrivibile. La differenza con <details> è esattamente questa: **hidden è presentazione, open è comportamento**.

## Le schede al posto della pila

.elenco-mazzi sotto i 46rem è a colonna singola. Tre mazzi impilati, ognuno con carosello ed elenco carte, sono tre schermate di scorrimento. Il rimedio era già nel progetto, un livello più in basso: <mazzo-generato> mostra una categoria per volta (Pokémon / Energie / Carte speciali) con un role="tablist", e il commento che lo accompagna dice perché.

Alzarlo di un livello — una scheda per mazzo — è la stessa idea applicata allo stesso problema in scala maggiore. Sopra i 46rem le schede spariscono e la griglia rimette i mazzi affiancati, perché lì il confronto fra mazzi è possibile e le schede lo toglierebbero:

```
@media (min-width: 46rem) {  
  .mazzi-schede { display: none; }  
  .elenco-mazzi mazzo-generato[hidden] { display: block; }  
}
```

Stessa soglia della griglia. Due soglie diverse avrebbero prodotto una finestra di larghezze in cui si vede una scheda sola in due colonne.

## Un dato che c'era e non si vedeva

I mazzi costruiti a mano si salvano con opzioni.personalizzato: true da sempre. Non lo mostrava nessuno: nell'elenco, un mazzo del wizard e uno costruito a mano erano indistinguibili. Ora è una pastiglia sulla card e un filtro sopra l'elenco — che compare **solo** se in casa ci sono entrambe le specie, altrimenti sarebbero tre pastiglie che non tolgono mai niente.

Vale la pena notare cosa **non** è servito: nessuna migrazione di schema, nessun campo nuovo, nessun VERSIONE\_DB da incrementare. Il dato era già lì. Prima di aggiungere un campo, conviene sempre chiedersi se l'informazione non sia già salvata sotto un altro nome.

## Una nota sul misurare invece di supporre

La prima versione della card metteva anche una barra di forza per mazzo, riuscendo .forza-barra della scheda grande. Sembrava un miglioramento. Misurata:

|                                   | altezza per mazzo |
|-----------------------------------|-------------------|
| riga vecchia (flex che va a capo) | ~75 px            |
| card con le barre                 | 133 px            |
| card senza barre                  | 94 px             |

Le barre costavano una riga e non dicevano niente che i numeri accanto non dicessero già. **Il rimedio allo scorrimento stava diventando la sua causa.** Sono state tolte, e il commento nel CSS dice perché, così a nessuno venga voglia di rimetterle.

## Domande di verifica

1. spezzaFrammento('#mazzi/2026-07-28T19:22:27.878Z') deve restituire l'id intero. Cosa sarebbe successo con pulito.split('/') e [nome, parametro]? E con split('/', 2)? Prova a scrivere l'id che rompe ciascuna delle due.
2. mostraStato('risultato') controlla if (!pianoCorrente) location.hash = 'mazzi'. Descrivi la sequenza esatta di azioni dell'utente che porta lì, e spiega perché quel caso non può esistere per #mazzi/<id>.
3. Nel CSS di stampa, mazzo-generato[hidden] { display: block !important } funziona, mentre l'equivalente per <details> no. Riscrivi la spiegazione in una frase sola, senza usare le parole "hidden" e "details".

4. `.azioni:not(:last-child) { margin-block-end: var(--spazio) }`. Perché la pseudo-classe invece di un margine incondizionato? Trova nel progetto i due punti in cui `.azioni` è l'ultimo figlio e di' cosa sarebbe successo.
5. Il pulsante “I miei mazzi” nel dettaglio viene ricreato a ogni `disegnaPiano()`. Con gli ascoltatori collegati uno per uno all'avvio, il pulsante *appariva* ma non faceva niente. Che tipo di difetto è — funzionale, di prestazioni, di accessibilità? E perché non se ne accorge nessun test che non apra un browser?